

- Elastic Agent Builder
- ES|QL deterministic tools
- Elastic Workflows
- Observability traces
- Amazon Bedrock + Sarvam

THE PROBLEM

In any lending or payments business, money moves across three systems that must agree: the internal **event-sourced ledger**, the **PSP / bank settlement file**, and the **disbursal pipeline** that actually pushed the funds. When these disagree, it is a **reconciliation break**: a disbursal marked success with no settlement credit, a double debit, a fee mismatch, a stuck mandate. Today an operations analyst investigates each break by hand. They pull ledger entries, cross-check the bank statement, trace the payment logs, and write up a root cause. This takes **hours per break**, does not scale on high-volume days, and the write-up is only as trustworthy as one tired human at 11pm. Breaks tie up settlement cash, delay customer refunds, and create audit exposure with the regulator.

THE SOLUTION

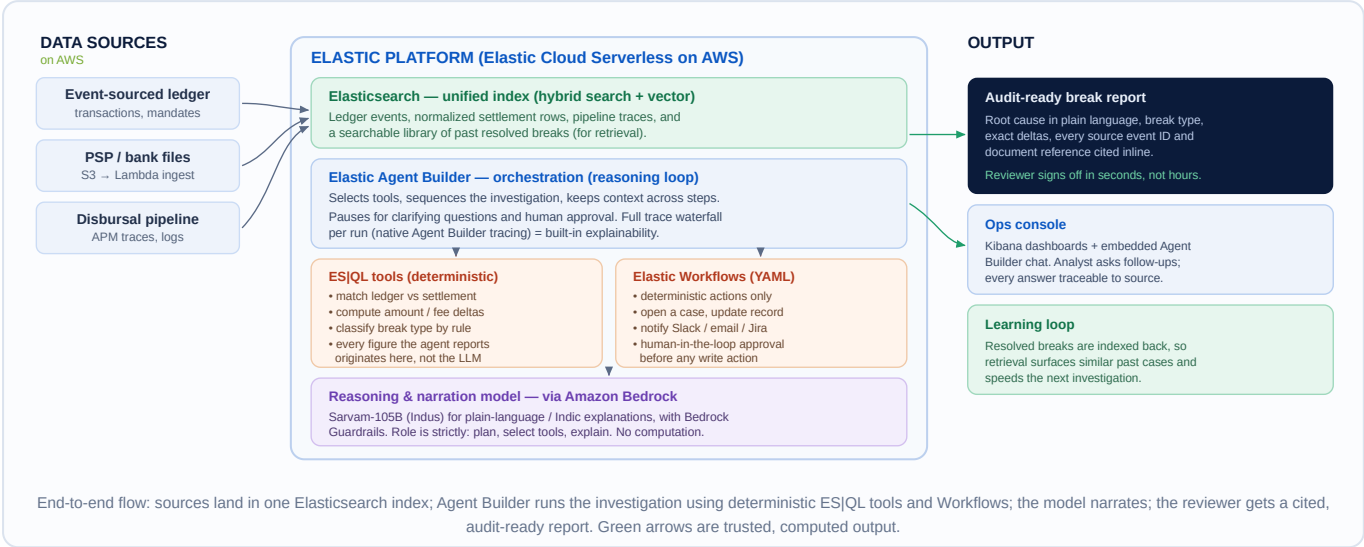
LedgerLens is a multi-step agent, built on **Elastic Agent Builder**, that investigates a reconciliation break end to end and produces an **audit-ready explanation** a finance or compliance reviewer can sign off in seconds. It reconstructs the truth of what happened to the money from the event-sourced ledger, matches it against the settlement file, correlates the disbursal pipeline traces, classifies the break, and narrates the finding in plain language with every supporting figure and document ID attached.

CORE DESIGN PRINCIPLE — THE TRUST STORY

The LLM never touches a number. Every amount, count, and match decision is computed by a deterministic ES|QL tool and passed through verbatim. The model only selects tools, orders the investigation, and explains the finished result. Nothing financial is ever generated by the model.

This is the same architecture that won 1st place at ClickHouse India's Click-a-thon (code ran the investigation, the model narrated). It maps directly onto how the Elastic Agent Builder hackathon winners scored: PHAROS won by keeping all statistical computation inside ES|QL and using the model only to narrate.

ARCHITECTURE



HOW EACH ELASTIC AND AWS CAPABILITY IS USED (MAPPED TO THE JUDGING RUBRIC)

CAPABILITY	ROLE IN LEDGERLENS	SCORES ON
Elastic Agent Builder	Multi-step reasoning loop, tool selection, clarifying questions, per-run trace waterfall for explainability.	Innovation (bonus marks), Technical Impl.
Custom ES QL tools	All money math and break classification as parameterized, rule-enforced queries. The deterministic core.	Technical Implementation, Impact
Elastic Workflows	Deterministic side effects (open case, notify, update) with human-in-the-loop approval before writes.	Technical Implementation, UX
Elasticsearch + hybrid search	Single substrate for ledger, settlement, traces, and a retrievable library of past resolved breaks.	Technical Implementation
Elastic Observability	Disbursal-pipeline APM traces and logs are the evidence the agent correlates to explain failures.	Impact, Technical Implementation
AWS (Bedrock, S3, Lambda)	Bedrock serves the reasoning model with Guardrails; S3 + Lambda ingest raw PSP/bank files; runs on Elastic Cloud on AWS.	Technical Implementation
Sarvam-105B (Indus)	Narration model, plain-language and Indic-ready explanations for Indian ops and compliance teams.	Impact, UX, sponsor fit

WHY THIS WINS, AND WHY THIS BUILDER

Measurable outcomes we will demo on a synthetic but realistic dataset of seeded breaks:

hrs → secs

Per-break investigation time, analyst manual vs agent

100%

Reported figures traceable to a source event ID

0

Numbers generated by the LLM, by construction

Judged on measurable performance and reliability, this design leads with a number that matters to a regulated business: zero hallucinated figures. That is a claim most agent submissions cannot make.

- **Real, costly, universal problem.** Every lending and payments company runs reconciliation and every one of them bleeds hours on breaks.
- **Deterministic by construction.** Reconciliation is matching, which is math, so keeping it in ES|QL is the natural design, not a bolt-on.
- **Builder credibility.** I ship this class of system in production: event sourcing, CQRS, distributed locking for dedupe, and the disbursal and mandate services this agent investigates. I can defend every technical claim in the Grand Finale.
- **Proven pattern.** The deterministic-execution, model-narrates design already won 1st at ClickHouse India's Click-a-thon and matches how the Elastic Agent Builder hackathon winners scored.

SCOPE FOR THE HACKATHON WINDOW, AND THE FINALE DEMO

In scope (what ships and runs)

- Ingestion of a synthetic ledger, a matching PSP settlement file, and disbursal traces into one Elasticsearch index.
- A working Agent Builder agent with three to four custom ES|QL tools covering match, delta, and rule-based break classification.
- One Elastic Workflow that opens a case and notifies, gated by a human approval step.
- A retrieval tool over past resolved breaks, plus the Kibana ops console with embedded chat.

Deliberately out of scope

- Live bank or PSP connections. We use realistic seeded data so results are reproducible for judging.
- Automated remediation of funds. Every write stays behind human approval by design, which is the correct posture for regulated money movement.

Live demo flow (3 minutes)

- **1.** Show a day's settlement with a seeded break: a disbursal marked success, no matching credit in the bank file.
- **2.** Trigger LedgerLens. It runs the ES|QL match tool, computes the exact missing amount, and classifies the break type by rule.
- **3.** It correlates the disbursal APM trace and finds the failure point in the pipeline.
- **4.** It returns the audit-ready report: root cause in plain language, every figure cited to a source event ID.
- **5.** Open the Agent Builder trace waterfall to prove the model computed nothing. Then approve the workflow that opens the case.

The single sentence a judge remembers: **the model explains, ES|QL decides, and every number is traceable.**